arXiv:2611.14210v1 [cs.LG] 10 Nov 2026

Tree-Ising: Repairing Language Models with Bit-Level Ising Edits

Hadi Salloum[1] Ammar Ali[2]

[1]QDeep, Riyadh, Saudi Arabia [2]LEAD (Lab for Efficient AI and Data), ITMO University

Abstract

Weight-space programming, introduced in recent MIT work by Takehi et al., enables a frontier LLM agent to read a network's weights and write a repair. Their GPT-6 Astra agent corrected the reasoning loops of Qwen3.5-0.8B without gradients, utilizing 158 agent turns, 89.7 million tokens, and 50 forward-pass requests. Our method, TREE-ISING, casts this repair as a sparse discrete optimization problem over the model's native BF16 weights. We leverage forward statistics to define an objective that weakens a loop direction, strengthens a concluding direction, and preserves the layer's output on productive tokens. Within a state-tree representation, where lossless BF16 storage acts as a special case, the admissible edits consist of sign flips and exponent shifts that preserve all shared group exponents and mantissas. We decompose this objective exactly into independent QUBO (Ising) problems, one per output row, which we solve on a GPU using simulated annealing or the quantum-inspired simulated bifurcation method. Evaluated on the two layers targeted by the agent, TREE-ISING achieves 54.6% on MATH-500, outperforming the agent's edit (51.7%) under the same evaluation protocol, while the base model scores only 7.9%. Furthermore, the simulated bifurcation solver yields a statistically indistinguishable repair at 55.1%. Crucially, both solvers outperform the agent edit across all five math benchmarks, with statistically significant improvements on MATH-500 and GSM8K after correcting for multiple comparisons. Moreover, our approach modifies 13% to 17% fewer weights and reduces BF16 bit flips by a factor of 2.2 to 2.3. It requires only four forward-pass requests and zero LLM tokens, with the annealing recipe costing approximately $0.90 in GPU time. Extending this to Qwen3.5-2B, where layers and dose are selected using only the base model, the same recipe increases greedy GSM8K accuracy from 21.6% to 69.4%. Finally, as a proof of concept, we execute an eight-variable repair subproblem using QAOA on an IBM Heron processor.



Figure 1: Two ways to repair a looping model. (A) Qwen3.5-0.8B loops in thinking mode and rarely finishes. (B) An LLM agent reads the weights, requests forward passes and writes a dense fix. (C) TREE-ISING finds the fix by optimization: one QUBO per output row, solved by GPU annealing with sign flips and exponent shifts inside each group's state tree. Bottom: MATH-500 accuracy (base model as reported by Takehi et al. [55]; agent and TREE-ISING under identical evaluation, Table 3), bits written (Table 5), cost of the agent's discovery run and of one run of our fixed recipe (Table 4), and transfer without a reference edit (Table 7).

1 Introduction

Weight-space programming has recently emerged as a compelling paradigm in model editing. Takehi et al. [55] at MIT investigated whether a language model can program the weights of another network: reading its parameters, diagnosing

a failure, and writing a fix directly into the weights, with an LLM agent acting as the programmer. Their most prominent demonstration targets a common failure mode in small reasoning models. In thinking mode, Qwen3.5-0.8B frequently falls into a short cycle of repeated text, continuing until the token budget is exhausted; this occurs on 93% of MATH-500 problems under a 32,768-token cap, limiting its accuracy to merely 7.9%. A GPT-6 Astra agent, provided with the checkpoint, practice problems, and a budget of 50 forward-pass requests, traced these loops to the MLPs of layers 15 and 19 and rewrote 64 columns in each of their output projections. Consequently, MATH-500 accuracy rose to 48.1% without a single gradient step. While this result proves that weights can be programmed, it also highlights the substantial price of doing so with an agent: 158 turns, 89.7 million tokens from a frontier model, 50 forward-pass requests, and 15.6 hours of compute for a single edit (Table 4). Crucially, every step of this process relies on judgment calls made in natural language, offering no guarantee that the resulting edit is the optimal one available. In this work, we cast the repair as a sparse discrete optimization problem over the stored weights. The evidence gathered by the agent—forward passes on looping answers versus successful ones—can be formalized as a mathematical objective, where an edit is defined as a discrete choice among changes to stored bits. We build this formulation on state trees [3], a general representation of neural networks in which every weight is a path of discrete decisions through a tree, and weights sharing the same decision share the recording node. Lossless BF16 storage is a special case of this representation, where each group of 64 weights shares its exponent as a node; this is the specific case we utilize. Our formulation, which we call TREE-ISING, comprises five key steps (Figure 2):

1. **Represent the weights as state trees.** In the lossless BF16 state tree, multiplying a weight by a power of two or flipping its sign alters only its 4-bit state and sign bit. Moves that preserve each group's maximum leave the shared exponent untouched. Preselecting one such move per candidate weight reduces the edit to a series of binary decisions.
2. **Select the layers.** A single forward scan of the base model evaluates how strongly each MLP writes along the loop direction, automatically identifying the layers to edit without requiring an agent.
3. **Define the objective.** Forward statistics yield a loop direction and a concluding direction. The objective is to minimize the layer's projection along their combination on loop tokens, preserve the output on productive tokens, and restrict the total number of modified weights.
4. **Formulate as an Ising model.** For fixed statistics and preselected moves, the objective decomposes exactly into independent subproblems, one per output row. Each row subproblem is formulated as a dense QUBO, equivalently an Ising model.
5. **Solve the optimization.** A GPU solves all row problems for a given layer simultaneously using simulated annealing or simulated bifurcation, a quantum-inspired Ising solver. Because these are native Ising models, they also map directly onto quantum annealers and QAOA.

Evaluated on the two layers targeted by the agent, TREE-ISING achieves 54.6% on MATH-500, outperforming the agent's edit (51.7%) under an identical evaluation protocol, while the base model scores only 7.9% in the evaluation of Takehi et al. [55]. Notably, it outperforms the agent edit across all five math benchmarks tested, with statistically significant gains on MATH-500 and GSM8K after correcting for multiple comparisons (Tables 1 and 3). Furthermore, replacing the annealer with simulated bifurcation [17, 18] yields an equally effective repair, achieving 55.1% on MATH-500 with 4% fewer modified weights, demonstrating that the results do not hinge on a single solver. Crucially, this approach requires only 4 forward-pass requests instead of 50, reducing forward-pass compute by a factor of 34 and eliminating the need for LLM tokens entirely. The entire recipe executes in 31 minutes at a cost of approximately $0.90. Moreover, the resulting edit is substantially smaller: it modifies 13% fewer weights, reduces BF16 bit flips by a factor of 2.2, cuts state-tree bit changes by a factor of 4, and, by construction of its move set, leaves all shared exponents and mantissas untouched. It is important to clarify the scope of this comparison. The agent incurred a significant cost to discover the failure, identify the direction, and select the layers. In contrast, our recipe was designed after analyzing its public logs; on Qwen3.5-0.8B, it reuses the agent's two selected layers and calibrates its dose based on the agent's edit. Thus, the cost figures compare the exhaustive process of finding a fix against the application of a fixed, optimized recipe. Our primary aim is to achieve a superior repair through fewer weight modifications. For context, a dense closed-form solution of the same objective, without sparsity or discrete moves, reaches similar accuracy on our practice screen but modifies 96% of the weights in both matrices—62 times as many as TREE-ISING (Section 4.5). Extending this recipe

Table 1: TREE-ISING against the LLM agent on Qwen3.5-0.8B, at a glance. Same model, same two layers, same evaluation harness; accuracy uses four completions per MATH-500 problem. Cost compares the agent's discovery run with one application of our fixed recipe, which reuses the agent's layers and dose (Section 4.3). Zero shared-exponent and mantissa writes hold by construction of the move set. TREE-ISING is solved by annealing here; simulated bifurcation gives an equivalent edit (55.1% on MATH-500, 108,327 changed weights). Details in Tables 3 to 5.

		Agent edit [55]	TREE-ISING (ours)	TREE-ISING advantage
Quality	MATH-500 accuracy (%)	51.7	**54.6**	+2.9 points, $p \approx 0.003$
	Benchmarks ahead (of 5)	–	**5**	2 significant after correction
	MATH-500 answers cut at cap (%)	31.6	**23.9**	1.3× fewer
Cost	Forward-pass requests	50	**4**	12.5× fewer
	Forward-pass GPU time	12.2 h	**21.8 min**	34× less
	LLM tokens	89.7M	**0**	none needed
	Wall clock	15.6 h	**31 min**	30× faster
	Cost	$312	**$0.90**	about 350× cheaper
Edit	Changed weights	130,188	**112,630**	13% fewer
	BF16 bit flips	621,388	**278,767**	2.2× fewer
	State-tree bit flips	720,586	**181,062**	4.0× fewer
	Shared-exponent writes	1,261	**0**	none (by design)
	Mantissa writes	129,668	**0**	none (by design)

to Qwen3.5-2B demonstrates that the agent is not strictly necessary. With layers selected by the forward scan and the dose calibrated from the base model alone, the method raises greedy GSM8K accuracy from 21.6% to 69.4%.

Contributions. The contributions of this work are summarized as follows:

- We propose a sparse discrete formulation for model repair on state trees, introducing tree-local moves, a forward-only objective, and an exact decomposition into independent row Ising problems for fixed statistics and preselected moves (Section 3).
- We demonstrate that on Qwen3.5-0.8B, TREE-ISING produces an edit that is more accurate than the agent-based repair of Takehi et al. [55], while modifying fewer weights and bits, at a small fraction of the forward passes, time, and computational cost (Section 4).
- We show that this recipe transfers effectively to Qwen3.5-2B without any reference edit, achieving a 47.8-point gain in GSM8K accuracy under greedy decoding (Section 5).

Finally, as a supplementary proof of concept, we successfully execute an eight-variable repair subproblem using QAOA on an IBM Heron quantum processor (Section 6).

2 Background

Reasoning loops. We define an answer as *cut* when the generation process reaches the maximum token budget before the model produces its final conclusion. In the models evaluated in this work, nearly all cut answers exhibit looping behavior: within the final 2,000 tokens, the majority of 8-grams have already appeared earlier in the same sequence. While repetition in neural text generation has been extensively studied through decoding heuristics and modified training objectives [22, 58, 59], and recent reports on reasoning models analyze how extended thinking traces interact with decoding configurations [12, 48], we frame looping as a property of the network's weights. This perspective treats the failure mode as an artifact that a direct weight edit can resolve.

Weight-space programming. Takehi et al. [55] enabled a frontier LLM agent to read the parameters of Qwen3.5-0.8B and execute forward passes on practice problems. Within a budget of 50 requests, the agent identified a specific direction

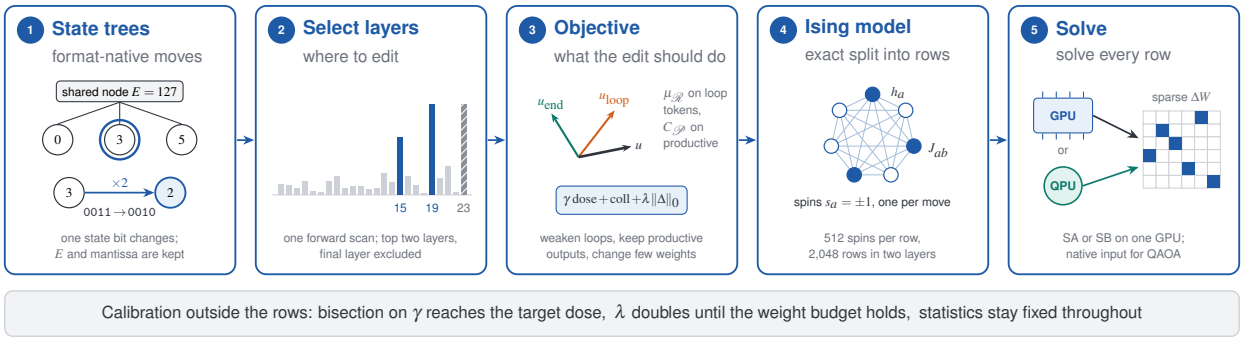


Figure 2: TREE-ISING in five steps. (1) In the lossless BF16 state tree, admissible moves keep the group's shared exponent and the mantissa; multiplying a weight by two changes one stored state bit. One move is preselected per candidate weight, so the edit becomes binary. (2) One forward scan of the base model scores each MLP's write along the loop direction and selects layers 15 and 19 (Figure 3). (3) Forward statistics define the dose along u, the collateral change on productive tokens and a sparsity cost. (4) For fixed statistics the objective splits exactly into one dense Ising model per output row. (5) GPU simulated annealing or simulated bifurcation solves all rows of a layer at once; the same models are the native input of quantum annealers and QAOA.

in the residual stream that remains active during loops, traced this behavior to the MLP output projections (`down_proj`) of layers 15 and 19, and applied a structured modification to 64 columns in each matrix. The comprehensive logs released by the authors [54] detail every interaction, token count, and forward pass, allowing us to precisely quantify the computational cost of the procedure (Section 4.3). We denote the resulting checkpoint as the *agent edit*. Crucially, the agent's workflow encompasses three distinct phases, which our formulation renders mathematically explicit: identifying the target layers, defining the optimization objective, and searching for the precise weight modifications required to achieve it.

3 Method: the repair as a sparse Ising problem

TREE-ISING edits frozen MLP output projections $W \in \mathbb{R}^{d\times n}$, where d denotes the number of output rows and n the number of input neurons. We cast this edit as a discrete optimization problem comprising five key steps (Figure 2): representing the weights and admissible moves (Section 3.1), selecting the target layers (Section 3.2), defining the optimization objective (Section 3.3), formulating the problem as independent Ising models (Section 3.4), and solving them (Section 3.5).

3.1 Step 1: a QUBO-friendly representation of the weights

State trees. State trees [3] provide a general representation for neural networks. Every weight is defined as a path of discrete decisions from the root of a tree to a leaf, and weights that share the same decision share the node that records it. This representation seamlessly unifies low-bit compression and exact storage. Lossless BF16 storage is a specific instance of this framework, and it is the one we employ. A BF16 number consists of one sign bit, an 8-bit exponent e, and a 7-bit mantissa [31]. The lossless BF16 state tree groups 64 consecutive weights within a row and stores their maximum exponent E once as a shared node. Each individual weight then stores a 4-bit state $D = E - e \in \{0,\dots,14\}$ and a leaf containing its sign and mantissa; state 15 designates a rare escape whose exponent is maintained in a side list. This instantiation reproduces every weight exactly, cleanly separating what a group shares (its exponent) from what each weight owns (its offset, sign, and mantissa), mirroring the structure of block floating-point and microscaling formats in their lossy counterparts [13, 51]. Because every stored quantity is a discrete decision, an edit is fundamentally a change of decisions.

Tree-local moves. We restrict each weight to be multiplied by a factor from the set

$$\mathscr{F} = \left\{-1,\ 2,\ \tfrac{1}{2},\ 4,\ -2,\ -4,\ -\tfrac{1}{2},\ \tfrac{1}{4}\right\} \tag{1}$$

or to remain unchanged. For a standard weight with state D, a factor with $k = \log_2 |f|$ shifts the state to $D-k$ and flips the sign if $f < 0$. A move is deemed admissible only if it preserves the group's shared exponent and generates no escape. Growth ($k > 0$) requires $D \geq k$, ensuring no weight exceeds the group maximum; shrinkage ($k < 0$) requires $D \geq 1$ and $D - k \leq 14$, guaranteeing that the group maximum is never reduced and no weight falls outside the 4-bit range. Sign flips are universally admissible. Every admissible move is exact in BF16, preserves the mantissa, and alters at most the sign bit and the 4-bit state of a single weight. For instance, the weight 0.125 in a group with $E = 127$ has $D = 3$; doubling it yields 0.25 with $D = 2$, resulting in a change to a single stored bit.

Binary formulation and the role of the state tree. The binary nature of the formulation arises from preselecting one move per candidate weight (Section 3.4): the sole remaining decision is whether to apply it, rendering the edit a vector of binary variables where every subsequent quantity is a polynomial of degree at most two, conforming to the QUBO format. Multiplying by a power of two is also exact in standard BF16, meaning the tree itself does not create the binary structure. Rather, its contribution lies in providing a format-native move set and its corresponding accounting. Because admissible moves never alter the group maximum, the shared node is never rewritten, and the number of stored bits modified by an edit can be directly read from the tree. The guarantee that the edit writes no shared exponent and no mantissa therefore holds by construction, rather than as an empirical observation.

3.2 Step 2: selecting the layers

Forward statistics. The base model generates an answer for each practice problem once, subject to an 8,192-token cap. Reference answers denote correct, completed traces, while a trace is classified as looping if it reaches the cap and more than 80% of the 8-grams in its tail repeat. Teacher-forced passes over these texts record, for every layer, the residual stream $h(t)$ after the layer, the output $m(t)$ of its MLP, and the input $a(t)$ of its output projection, across the four token sets defined in Table 2.

Table 2: Token sets used by the layer scan and the objective. All sets come from the base model's own answers to practice problems.

Set	Role	Definition
$\mathscr{R}$	loop	In a looping trace, find the first 256-token window in which at least 80% of 8-grams repeat, after at least 300 generated tokens. In the next 1,000 positions, take the final token of every repeated 8-gram.
$\mathscr{P}$	protected	The 2,000 to 50 tokens before each loop onset, and every generated token of correct completed traces.
$\mathscr{Z}$	concluding	The last 64 thinking tokens before `</think>` in a correct trace with at least 200 thinking tokens.
$\mathscr{M}$	continuing	The earlier thinking tokens of the same correct traces.

The mean residuals after layer ℓ yield two unit directions and their combination,

$$u_{\ell,\text{loop}} \propto \bar{h}_{\ell,\mathscr{R}} - \bar{h}_{\ell,\mathscr{P}}, \qquad u_{\ell,\text{end}} \propto \bar{h}_{\ell,\mathscr{Z}} - \bar{h}_{\ell,\mathscr{M}}, \qquad u_\ell \propto u_{\ell,\text{loop}} - \beta\, u_{\ell,\text{end}}, \tag{2}$$

with $\beta = 0.5$. Reducing the projection of a layer's output onto u_ℓ weakens the loop and encourages the model to conclude.

Layer scan. The write score of layer ℓ quantifies how much more its MLP pushes the residual stream along u_ℓ on loop tokens compared to productive ones,

$$s_\ell = u_\ell^\top \left(\bar{m}_{\ell,\mathscr{R}} - \bar{m}_{\ell,\mathscr{P}}\right). \tag{3}$$

Our selection rule targets the two highest-scoring layers, excluding the final layer, whose MLP writes directly into the unembedding. On Qwen3.5-0.8B, three layers stand out significantly (Figure 3, top): layer 23 (the final layer), and layers 19 and 15, each scoring at least twice as high as any other layer. Consequently, the rule selects layers 15 and 19, exactly matching the agent's choice. We established this exclusion rule after observing these scores and subsequently applied it unchanged to Qwen3.5-2B, where it again selects layers 15 and 19 (Figure 3, bottom). In contrast, a gradient-based attribution, which scores layers by the first-order effect of removing their MLP output on the log-probability of continuing a loop, ranks layers 14, 21, 18, and 16 highest, failing to recover the agent's choice.

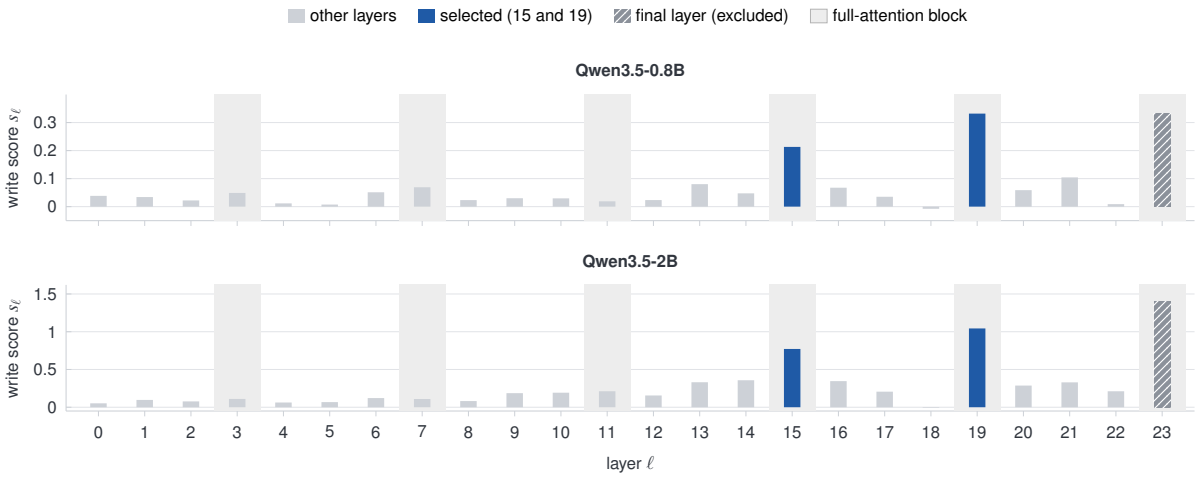


Figure 3: Forward layer scan of both models. Write score of every MLP along the loop direction, on loop tokens relative to productive tokens (Equation (3)). Shaded columns mark full-attention blocks; all other blocks use linear attention. The rule "two highest layers, excluding the final layer" was fixed on Qwen3.5-0.8B, where it returns the agent's choice, and selects layers 15 and 19 again on Qwen3.5-2B. In both models the three highest layers are full-attention blocks, so the agreement may follow from the shared architecture.

Both models utilize a hybrid layout where every fourth block (layers 3, 7, 11, 15, 19, and 23) employs full attention, while the others use linear attention. In both models, the three highest-scoring layers are full-attention blocks (shaded in Figure 3). The agreement between the two models may therefore reflect their shared architectural design rather than providing independent evidence that the scan identifies the optimal layers. Nevertheless, this alignment is suggestive in itself, since only full attention can copy exactly from distant positions in the context, which is precisely what a loop requires. Evaluating the scan on a model without this specific layout remains a direction for future work.

3.3 Step 3: the objective

For a given edited layer, we drop the layer index ℓ and denote its target direction as u. The objective requires the mean input on loop tokens, $\mu_{\mathscr{R}} = \mathbb{E}_{t\in\mathscr{R}}[a(t)]$, and the second moment of inputs on protected tokens, $C_{\mathscr{P}} = \mathbb{E}_{t\in\mathscr{P}}[a(t)a(t)^\top]$. For a fixed input $a(t)$, an edit Δ alters the layer output by $\Delta a(t)$. We quantify the effect of the edit along u on loop tokens, termed its *dose*, and its perturbation on protected tokens, termed its *collateral*:

$$\mathrm{dose}(\Delta) = u^\top \Delta\, \mu_{\mathscr{R}}, \qquad \mathrm{coll}(\Delta) = \mathbb{E}_{t\in\mathscr{P}}\left\|\Delta a(t)\right\|^2 = \sum_{r=1}^{d} \Delta_{r:} C_{\mathscr{P}} \Delta_{r:}^\top. \tag{4}$$

This collateral term corresponds to the layer-output reconstruction error commonly employed in pruning and quantization [16, 19]. For scaling factors $\gamma > 0$ and $\lambda \geq 0$, the edit minimizes

$$\mathscr{E}(\Delta) = \gamma\,\mathrm{dose}(\Delta) + \mathrm{coll}(\Delta) + \lambda\,\|\Delta\|_0 \qquad \text{over edits constructed from admissible moves.} \tag{5}$$

The parameter γ is calibrated to ensure the edit achieves a target dose, while λ restricts the edit to a specific budget of changed weights (Section 3.5). The target is defined relative to the layer's own write along u on loop tokens, $d_{\mathrm{target}} = -\tau\left|u^\top W \mu_{\mathscr{R}}\right|$. On Qwen3.5-0.8B, we set the target to 1.6 times the dose of the agent's edit along the same u, a multiplier fixed during development prior to the dose sweep in Table 6; this corresponds to $\tau = 6.7$ for layer 15 and $\tau = 6.1$ for layer 19 (the achieved doses are 6.8 and 6.2 times the own write, respectively, as shown in Table 9). The comparison with the agent is therefore not strictly dose-matched. For a novel model, we use $\tau = 6.5$, ensuring the entire objective is derived from the base model alone. Crucially, the formulation relies exclusively on forward passes, requiring no gradients or training.

3.4 Step 4: exact decomposition into row Ising models

Each of the three terms in Equation (5) decomposes into sums over the output rows: the dose is linear in Δ, the collateral is a sum of row-wise quadratic forms, and the sparsity penalty counts nonzeros row by row. For fixed statistics and penalties, the problem therefore separates exactly into one independent problem per row. The weight budget couples the rows only through their aggregate total, which is managed by the outer adjustment of λ.

Within each layer, we retain the K columns exhibiting the largest scores $(u^\top W_{:j})\,\mu_{\mathscr{R},j}$, corresponding to the neurons whose current write along u on loop tokens is most significant. Because this score is independent of the row, every row of a given layer utilizes the same K columns. For each retained weight (r, j_a), we preselect the admissible factor f_a that yields the lowest single-weight energy at the current γ and λ. We define $\delta_a = (f_a - 1)W_{rj_a}$ and let $x_a \in \{0,1\}$ act as a switch for the move. The row problem is then formulated as the QUBO

$$\mathscr{E}_r(x) = \sum_{a=1}^{K} h_a x_a + \sum_{a\neq b} J_{ab} x_a x_b, \qquad h_a = \gamma u_r\,\mu_{\mathscr{R},j_a}\,\delta_a + C_{\mathscr{P},j_a j_a}\,\delta_a^2 + \lambda, \qquad J_{ab} = \delta_a\,\delta_b\,C_{\mathscr{P},j_a j_b}. \tag{6}$$

The linear field h_a aggregates the dose gained, the self-collateral, and the sparsity cost of a single move; the coupling J_{ab} represents the collateral generated jointly by two moves. By substituting spins $s = 2x - 1$, the same energy is expressed as an Ising model,

$$\mathscr{E}_r(s) = \text{const} + \sum_a H_a s_a + \sum_{a<b} I_{ab} s_a s_b, \qquad H_a = \tfrac{1}{2}h_a + \tfrac{1}{2}\textstyle\sum_{b\neq a} J_{ab}, \qquad I_{ab} = \tfrac{1}{2}J_{ab}, \tag{7}$$

where J is symmetric and enters Equation (6) through the ordered-pair sum. This decomposition is exact given the retained columns and the preselected moves; both are heuristics designed to shrink the search space, and we repeat the preselection whenever γ is updated. With $K = 512$, each edited layer of Qwen3.5-0.8B yields 1,024 dense Ising problems with 512 spins each, resulting in 2,048 problems across the two edited layers.

3.5 Step 5: solving

GPU annealing. Our default solver processes all rows of a given layer as a single GPU batch via simulated annealing [33]. The final solve executes 16 replicas per row, starting from the zero edit, for 300 sweeps of single-spin Metropolis updates in random order. The temperature decreases geometrically from 2 to 10^{-4} times the row's median $|h_a|$, and we retain the lowest-energy replica.

Simulated bifurcation. As an alternative solver, we employ discrete simulated bifurcation (dSB) [17, 18], a quantum-inspired method that integrates the classical dynamics of coupled bifurcating oscillators and reads spins from their signs. Each replica initializes near the zero edit; the pump ramps up over the first quarter of 1,200 steps of size 0.5, and the best sampled state of every trajectory, refined by single-flip descent in float64, is retained. Rows are processed in batches of 32, maintaining the same replica counts as in annealing. All other components, including the candidates, the preselected moves, and the calibration protocol, remain unchanged, ensuring the two solvers differ solely in how each row problem is searched.

Calibration. Two global scalars are calibrated prior to solving the individual row problems. The sparsity weight initializes at $\lambda_0 = 2\,\text{median}\,C_{\mathscr{P},jj}\,\delta^2$, representing the median single-weight collateral over the retained weights and the three basic moves $f \in \{-1, 2, \frac{1}{2}\}$; the candidate moves themselves always utilize all eight factors. At each value of λ, a geometric bisection over $\gamma \in [10^{-6}, 10^{6}]$, utilizing quick solves of 4 replicas and 60 sweeps, identifies the smallest γ whose edit reaches the target dose, and the final solve adopts that γ. If the layer subsequently exceeds its budget, λ is doubled and the γ search is repeated for at most six attempts, ensuring the dose target is re-met after every adjustment to λ. The statistics remain fixed throughout this process. Table 9 reports the achieved dose and edit count for every edited layer.

Alternative solvers. Crucially, the formulation in Steps 1 through 4 is entirely solver-agnostic. The row problems are standard Ising models and can be seamlessly handed to any Ising solver, ranging from greedy descent to quantum annealing [28, 30] or QAOA [15]. We examine the quantum route in Section 6.

4 Repairing Qwen3.5-0.8B: Comparison with the LLM Agent

4.1 Experimental Setup

Model and Edit. We apply our method to the `down_proj` matrices ($1{,}024 \times 3{,}584$) in layers 15 and 19 of Qwen3.5-0.8B [48], matching the layers targeted by the agent edit. We configure the optimization with $K = 512$ candidates per row, $\beta = 0.5$, and a strict budget of 65,536 changed weights per layer. The target dose is set to 1.6 times the dose of the agent edit along the direction u, corresponding to targets of 6.7 and 6.1 times the layer's own write for layers 15 and 19, respectively (Section 3.3); consequently, the two edits are not strictly dose-matched. Notably, the forward scan detailed in Section 3.2 selects these exact same two layers. However, the exclusion rule for this scan was finalized after observing the initial scores, and the entire recipe was designed retrospectively by analyzing the agent's public logs. We refer to this specific configuration as the TREE-ISING edit. Unless otherwise specified, this edit utilizes the simulated annealing solver; variants marked with "SB" denote the identical recipe solved via simulated bifurcation (Section 3.5). A complete list of hyperparameters and settings is provided in Section A.

Data. The forward statistics are derived from 384 practice problems released alongside the agent's evaluation harness, comprising 192 problems from the MATH training split and 192 from the GSM8K training split [6, 21]. An additional 128 held-out practice problems constitute the selection screen used for the ablation studies in Section 4.5. We note that early exploratory variants were evaluated on a fixed subset of 100 MATH-500 problems; consequently, the full MATH-500 results include problems encountered during the development phase, a limitation we address in Section 8.

Evaluation. We strictly adhere to the evaluation protocol established by the agent's authors [54]: generation is performed using vLLM [35] in thinking mode, with a temperature of 0.6, top-p of 0.95, top-k of 20, a presence penalty of 1.5, and a maximum of 32,768 new tokens. The final boxed answer following the `</think>` token is graded using Math-Verify [36]. We report performance across five benchmarks: MATH-500 [39], GSM8K [6], AMC23 [1], Minerva Math [37], and OlympiadBench [20]. The suffix @k denotes the number of completions generated per problem, with accuracy defined as the mean correctness across these completions. To ensure a fair comparison, both edited checkpoints are evaluated within the identical harness using the same random seeds, and paired statistical tests are conducted on a problem-by-problem basis.

4.2 Accuracy and Termination

Table 3 presents a direct comparison of the edits. Crucially, with either solver, TREE-ISING outperforms the agent edit across all five benchmarks. The performance gains on MATH-500 (with four completions per problem) and on GSM8K remain statistically significant even after applying a Bonferroni correction across all twelve comparisons with the agent edit (two solvers, six settings; $p < 0.05/12$ in all four cases). Furthermore, with simulated annealing, the improvement on OlympiadBench is nominally significant ($p = 0.045$), though it does not survive the strict correction. The remaining differences fall within the expected noise margin.

Two key observations clarify the evaluation harness. First, TREE-ISING surpasses the performance metrics originally reported by the agent's authors on four of the five benchmarks, with the sole exception of AMC23. Second, our measured MATH-500 score for the agent edit (51.7% accuracy with 31.6% of answers cut) aligns closely with an independent reproduction of the reconstructed checkpoint on the full MATH-500 dataset, which reported 51.0% accuracy with an identical 31.6% cut rate [32]. Consequently, the discrepancy with the originally reported 48.1% appears to stem from the initial evaluation setup rather than our harness.

Problems Never Used During Development. Early exploratory variants were screened on a fixed subset of 100 MATH-500 problems. On the remaining 400 problems, which were entirely unseen during development, TREE-ISING achieves 53.8% accuracy compared to 50.8% for the agent edit (with four completions per problem). This represents a paired difference of $+2.9 \pm 1.1$ points ($z = 2.62$, $p \approx 0.009$; 87 problems improved, 58 degraded), with the cut rate dropping from 31.2% to 24.0%. The SB edit scores 53.6%, maintaining a $+2.8 \pm 1.1$ point advantage over the agent edit ($p \approx 0.007$; 85 better, 52 worse). Notably, the 0.5-point lead of the SB edit over annealing on the full 500-problem

Table 3: Benchmark accuracy on Qwen3.5-0.8B (%). The suffix @k indicates the number of completions per problem. The upper block reproduces the numbers reported by Takehi et al. [55]; the lower block evaluates the edited checkpoints within a unified harness, with TREE-ISING solved by simulated annealing (SA) or simulated bifurcation (SB). The bottom rows detail the percentage of completions cut at the 32,768-token cap.

	MATH-500@1	MATH-500@4	GSM8K@1	AMC23@16	Minerva@4	OlympiadBench@1
Base model, reported	–	7.9	13.8	0.5	1.5	1.3
Agent edit, reported	–	48.1	67.1	23.4	25.4	21.8
Agent edit, our harness	51.2	51.7	65.6	22.0	24.2	21.4
TREE-ISING (SA)	54.0	54.6	70.5	22.7	**26.1**	**24.3**
TREE-ISING (SB)	**54.4**	**55.1**	**70.8**	**25.0**	25.6	24.0
Cut at cap (%)						
Agent edit, our harness	31.4	31.6	8.0	51.4	26.7	47.8
TREE-ISING (SA)	23.0	**23.9**	**3.1**	37.8	**18.1**	32.5
TREE-ISING (SB)	**22.8**	**23.9**	3.2	**36.6**	18.5	**31.9**

Paired comparisons against the agent edit. TREE-ISING (SA): MATH-500@4 $+2.9 \pm 1.0$ points, $z = 2.94$, $p \approx 0.003$ (better on 108 problems, worse on 73); GSM8K: 155 problems solved exclusively by TREE-ISING versus 90 by the agent edit ($p = 4 \times 10^{-5}$); OlympiadBench: 55 versus 35 ($p = 0.045$); MATH-500@1: 57 versus 43 ($p = 0.19$); AMC23: $+0.6 \pm 1.0$ (9 better, 7 worse); Minerva: $+1.9 \pm 1.2$ (47 better, 34 worse). TREE-ISING (SB): MATH-500@4 $+3.4 \pm 1.0$, $z = 3.47$, $p \approx 0.0005$ (113 better, 65 worse); GSM8K: 156 versus 87 ($p = 10^{-5}$); OlympiadBench: 55 versus 37 ($p = 0.08$); MATH-500@1: 52 versus 36 ($p = 0.11$); AMC23: $+3.0 \pm 1.5$ (16 better, 7 worse); Minerva: $+1.5 \pm 1.2$ (46 better, 44 worse). The base model reaches the token cap on 93% of MATH-500 completions. The agent's originally reported counts utilized two GSM8K and four OlympiadBench completions.

set originates entirely from the 100 development problems; for annealing, the performance gap on the held-out problems matches the overall gap.

The most pronounced difference between the edits lies in their termination behavior. TREE-ISING reaches the token cap less frequently across every benchmark. For instance, on GSM8K, the cut rate plummets from 8.0% to 3.1%, and on OlympiadBench, it drops from 47.8% to 32.5%. These accuracy gains are primarily driven by the model's newfound ability to successfully conclude its reasoning within the allocated token budget.

Solver Choice. Both solvers yield repairs of equivalent quality. Simulated bifurcation lands marginally closer to the target doses (-2.189 and -3.083 against targets of -2.183 and -3.056, whereas annealing reaches -2.225 and -3.075), while also exhibiting lower collateral in layer 15 and modifying 4% fewer weights (108,327 versus 112,630; see Tables 5 and 9). Crucially, no benchmark statistically separates the two edits: on MATH-500@4, the SB edit leads by a negligible $+0.45 \pm 0.9$ points ($p = 0.62$); on GSM8K, the two solvers uniquely solve 115 and 111 problems, respectively ($p = 0.84$); and the largest observed gap, $+2.3 \pm 1.3$ points on AMC23, remains statistically insignificant. This demonstrates that the performance gains stem fundamentally from the mathematical formulation itself, rather than from the tuning of a specific solver.

4.3 Cost of a Single Repair

Table 4 quantifies the computational expense of a single repair. The agent expended 158 turns and nearly 90 million tokens, with its 50 forward-pass requests alone consuming 12.2 GPU-hours. In stark contrast, TREE-ISING requires only 4 forward-pass requests: one generation pass over the practice problems, two teacher-forced passes for statistics collection, and one validation screen. These operations consume merely 21.8 GPU-minutes, representing a 34-fold reduction in forward-pass compute compared to the agent, and the subsequent annealing adds just 8.8 minutes. Consequently, our repair is approximately 30 times faster and 350 times cheaper, while requiring zero LLM tokens. Notably, the agent's preliminary runs with 10 and 25 forward passes, which failed to yield a functional fix, incurred an additional $270 at the same pricing tier.

It is critical to contextualize this comparison: it contrasts the exhaustive process of discovering a fix with the efficient application of a fixed recipe. The agent bore the computational burden of identifying the failure mode, the loop direction, and the target layers. Our recipe was designed retrospectively by analyzing its public logs, reusing its layer

Table 4: Computational cost of producing a single edit. This comparison contrasts the agent's exhaustive discovery run against a single execution of our fixed recipe, which reuses the agent's targeted layers and dose. Agent figures are derived from the released 50-pass logs [54] at list prices [47]; GPU time is priced at the median A100-40GB on-demand rate of $1.76 per hour [4]. A single forward-pass request accommodates up to 512 generations or 1,024 teacher-forced texts. Method development costs are excluded for both approaches.

	Agent Edit	TREE-ISING
Search procedure	LLM agent (GPT-6 Astra, maximal reasoning effort)	Simulated annealing on 2,048 row QUBOs
Agent turns and tokens	158 turns, 89.7M tokens (80.3M cached input, 2.9M output)	None
Gradients	None	None
Forward passes	50	4 (one generation, two statistics passes, one screen)
Model GPU time	12.2 h	21.8 min
Search time	1.8 h of agent responses	8.8 min of GPU annealing
Wall clock	15.6 h (20.3 h including a 4.7 h API outage)	**31 min** (30.6 min)
LLM API cost	$291 (up to $485 at long-context rates)	$0
GPU cost	$21	$0.90
Total	**$312**	**$0.90**

Table 5: Weight and bit modifications across both edited matrices. State-tree fields comprise the 8-bit shared node, the 4-bit state, the sign bit, and the 7-bit mantissa; modifying a shared node inherently alters the states of all other weights within its group. Neither edit alters escape membership, and changes confined to existing escape payloads are excluded from this count.

	Changed Weights	BF16 Bit Flips	State-Tree Bit Flips	Sign Flips	State Changes	Shared Exponents	Mantissa Changes
Agent Edit	130,188	621,388	720,586	19,299	146,981	1,261	129,668
TREE-ISING (SA)	112,630	278,767	181,062	57,704	99,952	**0**	**0**
TREE-ISING (SB)	**108,327**	**268,090**	**174,245**	55,666	**96,007**	**0**	**0**

selections, and calibrating the dose based on its final edit. Both columns explicitly exclude method development costs. Furthermore, Section 5 demonstrates the application of this recipe to a novel model without relying on any of these prior dependencies.

4.4 Edit Footprint

Table 5 details the precise structural modifications introduced by each edit. TREE-ISING modifies 13% fewer weights (affecting only 1.5% of the two matrices), reduces BF16 bit flips by a factor of 2.2, and cuts state-tree bit flips by a factor of four; the SB edit achieves an additional 4% reduction in size. Crucially, while the agent edit rewrites the mantissa of nearly every weight it touches and shifts 1,261 shared exponents, TREE-ISING performs neither operation, a guarantee enforced by the construction of its admissible move set. The most frequently applied moves are scaling by $\times 4$ and $\times(-4)$, followed by $\times 2$, $\times(-2)$, and sign flips. Consequently, a patch of this nature is exceptionally compact, mathematically exact, and entirely native to the stored format.

4.5 Ablation on the Objective and Continuous Edits

Table 6 systematically varies one parameter at a time on the practice screen. The most pronounced effect arises from the inclusion of the concluding direction. Without it ($\beta = 0$), the edit successfully suppresses the loop, but the model still fails to terminate, resulting in accuracy stagnating in the mid-40s with 53% of answers hitting the token cap. Incorporating this term, however, elevates the accuracy well above that of the agent edit. Similarly, restricting the admissible moves to merely sign flips and single doublings or halvings, or severely shrinking the weight budget, degrades performance. Doses ranging from $1.2\times$ to $2.0\times$ yield comparable results, and both annealing and simulated

Table 6: Ablation study on the practice screen (128 held-out problems, one completion each; the standard error of accuracy is approximately 4.4 points). The selected configuration utilizes $\beta = 0.5$, a 1.6× dose, the eight admissible moves from Equation (1), simulated annealing, and a budget of 65,536 changed weights per layer. Each row in the middle block isolates a single modified setting. The lower block solves the identical objective without sparsity or discrete moves, using the same direction u and dose, and subsequently rounds the result to BF16; $\bar{c}$ denotes the mean diagonal of $C_{\mathscr{P}}$.

Variant	Change	Accuracy (%)	Cut at cap (%)	Changed Weights	BF16 Bit Flips
TREE-ISING	Selected configuration	**57.0**	22.7	112.6k	279k
Loop only	$\beta = 0$	43.8	53.1	112.3k	277k
Weaker conclude term	$\beta = 0.25$	53.1	28.1	115.4k	285k
Stronger conclude term	$\beta = 1$	53.9	21.1	93.2k	232k
Lower dose	1.2×	57.8	16.4	77.4k	193k
Higher dose	2.0×	54.7	26.6	118.5k	301k
Basic moves	$f \in \{-1, 2, \frac{1}{2}\}$	50.0	22.7	122.7k	179k
Greedy solver	Zero temperature	53.9	18.0	112.0k	277k
Simulated bifurcation	SB in place of annealing	56.3	18.8	108.3k	268k
Smaller budget	28k weights per layer	49.2	24.2	45.6k	121k
Agent Edit	Released checkpoint	52.3	25.0	130.2k	621k
Continuous edits of the same objective					
Closed-form rank-one	$\Delta \propto -u\left((C_{\mathscr{P}} + 10^{-3}\bar{c}I)^{-1}\mu_{\mathscr{R}}\right)^{\top}$	57.8	12.5	7.02M	28.7M
Stronger ridge	$10^{-1}\bar{c}$ in place of $10^{-3}\bar{c}$	56.3	7.0	6.94M	26.6M
Directional ablation	$\Delta = -\alpha u u^{\top} W$	38.3	25.0	6.92M	26.1M

bifurcation outperform a greedy solver, although the screen lacks the resolution to distinguish differences of a few percentage points. The 1.6× dose was fixed during the initial development phase, prior to this dose sweep, which was subsequently executed around the selected configuration. While we did not reselect the configuration post-sweep, it is worth noting that the lower dose achieves a marginally higher score while modifying 31% fewer weights. No strictly dose-matched (1.0×) edit was evaluated.

Continuous Edits. If we remove the sparsity penalty and the discrete move constraints, Equation (5) admits a closed-form minimizer: the rank-one update $\Delta = -\frac{\gamma}{2} u (C_{\mathscr{P}}^{-1}\mu_{\mathscr{R}})^{\top}$, which structurally mirrors the updates employed by ROME [41]. When rounded to BF16 at an equivalent dose, this continuous solution scores 57.8% on the screen, matching TREE-ISING within the screen's margin of error, and reaches the token cap even less frequently. However, it fundamentally alters 96% of the weights across both matrices (7.0 million changes versus 112,630) and induces 103 times as many BF16 bit flips. A plain directional ablation [5] at the same dose scores a mere 38.3%. These continuous variants were evaluated exclusively on the screening set, not on the full benchmarks. Crucially, they demonstrate that the mathematical objective itself, rather than the discrete solver, drives the majority of the accuracy gains. Furthermore, they highlight that TREE-ISING achieves the accuracy of the optimal continuous solution while modifying 62 times fewer weights, with all changes written natively into the stored format.

Which Size Matters. We quantify the number of changed stored weights because the edit is applied as a direct patch to the stored checkpoint: the repaired model is served using ordinary weights, in place, with an entirely unchanged inference graph. From a purely descriptive standpoint, the rank-one edit appears smaller, requiring only two vectors (4,608 numbers per layer) compared to the approximately 56,000 (position, move) pairs per layer required by TREE-ISING. However, this compactness holds only if the edit is preserved as a separate rank-one adapter during inference. When written directly into the weights, the continuous edit rewrites 96% of the stored values and nearly every mantissa, necessitating the complete re-encoding of any compressed, deduplicated, or state-tree checkpoint. In stark contrast, a TREE-ISING patch leaves every unrelated stored bit entirely untouched, with each modification confined strictly to a single sign bit or a 4-bit state.

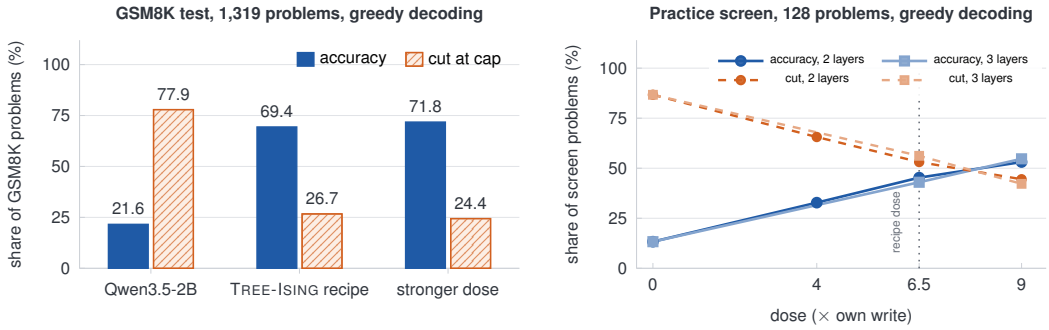


Figure 4: The recipe transfers to Qwen3.5-2B. Left: on all 1,319 GSM8K problems the unchanged recipe (layers 15 and 19, dose 6.5×) lifts accuracy from 21.6% to 69.4% and cuts the share of looping answers by two thirds; layers 14, 15 and 19 at 9× reach 71.8%. Right: on the practice screen, accuracy rises and the cut share falls steadily with dose, both for layers 15 and 19 and for layers 14, 15 and 19.

5 Transfer to Qwen3.5-2B Without a Reference Edit

In our evaluation of Qwen3.5-0.8B, TREE-ISING relied on the agent's selected layers and calibrated its dose based on the agent's final edit. In this section, we eliminate both dependencies. We apply the method to a novel model, autonomously selecting its target layers via the forward scan and calibrating the dose using the base-model rule defined in Section 3.3, while leaving the remainder of the recipe entirely unchanged.

The Failure Mode. Qwen3.5-2B [49] comprises 24 layers with `down_proj` matrices of dimensions 2,048 × 6,144. Under greedy decoding (temperature 0, thinking mode, 8,192-token cap), the base model correctly answers only 21.6% of the 1,319 GSM8K test problems (Table 7). The underlying cause mirrors the failure observed in the smaller model: 77.9% of its generations reach the token cap, and 1,018 of these 1,028 truncated answers exhibit looping behavior. Crucially, when the model does successfully terminate, it provides the correct answer 97.9% of the time. This indicates that the model possesses the requisite problem-solving capability but fails to halt its reasoning. We specifically selected greedy decoding for this evaluation because an initial probe on 200 GSM8K test problems revealed that the base model struggles most severely under these conditions; consequently, the recipe has not yet been evaluated under the model's recommended sampling settings or on the MATH-500 benchmark.

The Recipe. We first prompt the base model to answer the same 384 practice problems using greedy decoding, yielding a 16.9% success rate with 83.1% of the answers truncated. The forward scan subsequently ranks layers 23, 19, 15, and 14 highest (Figure 3, bottom). Applying our exclusion rule for the final layer isolates layers 15 and 19, precisely matching the pair selected for the smaller model. The forward statistics are derived from 275 looping and 65 correct traces. We set the target dose to 6.5 times each layer's own write, and allocate a budget of 200,000 changed weights per layer, representing approximately 1.6% of a single matrix and closely aligning with the 1.8% utilized for the smaller model. All remaining hyperparameters are identical to those detailed in Section 4. The entire pipeline, encompassing the layer scan and a single validation screen, executes in just 26 minutes on a single A100 GPU. Crucially, this process requires no LLM agent, no reference checkpoint, and no gradient computations.

Results. Applying the unchanged recipe elevates GSM8K accuracy from 21.6% to 69.4%, representing a substantial gain of 47.8 percentage points (Table 7). The edit successfully solves 669 problems that the base model fails to answer, while incorrectly altering the solution for 38 of the 285 problems the base model originally solved correctly (a 13% degradation on those specific instances). Notably, when evaluated on the 1,119 problems outside the initial 200-problem probe used to select the decoding setting, the performance remains consistent: 69.6% accuracy versus 21.6% for the base model (571 problems solved exclusively by the edit, 34 exclusively by the base model, and 208 by both). Furthermore, the proportion of truncated answers drops precipitously from 77.9% to 26.7%. While the accuracy on fully completed answers decreases slightly from 97.9% to 94.7%, it is important to note that these figures refer to different subsets of problems; the 38 lost problems confirm that the edit does perturb some correct base-model trajectories. Structurally, the

Table 7: Performance of Qwen3.5-2B on GSM8K. Evaluated across all 1,319 test problems using greedy decoding in thinking mode with an 8,192-token cap. The TREE-ISING recipe utilizes the scan-selected layers and the base-model dose rule without any modifications. Paired counts detail the number of problems solved exclusively by the edit, exclusively by the base model, and by both.

Model	Layers	Dose	Accuracy (%)	Cut at cap (%)	Only edit / only base / both	Changed weights	State-tree bit flips
Qwen3.5-2B	–	–	21.6	77.9	–	–	–
TREE-ISING recipe	15, 19	6.5×	**69.4**	26.7	669 / 38 / 247	292,845	473,524
TREE-ISING, stronger dose	14, 15, 19	9×	**71.8**	24.4	687 / 25 / 260	482,423	788,492

Statistical comparisons against the base model: McNemar $p < 10^{-100}$ for both edits. *Stronger dose versus the standard recipe:* 171 problems solved exclusively by the stronger edit and 140 exclusively by the recipe ($p = 0.09$). Crucially, neither edit alters a shared exponent, a mantissa, or escape membership. Total BF16 bit flips: 718,546 for the standard recipe and 1,178,678 for the stronger dose.

patch modifies a mere 1.2% of the weights across the two matrices and, consistent with our design, alters no shared exponents and no mantissas.

Dose Calibration. Reasoning loops prove more persistent under greedy decoding than under sampling, and the practice screen indicates that performance continues to improve with higher doses (Figure 4, right). Consequently, we also evaluated a dose of 9× alongside the inclusion of the next highest-scoring scan layer, layer 14. The best screened variant, targeting layers 14, 15, and 19 at a 9× dose, achieves 71.8% on GSM8K, a 50.2-point improvement over the base model. However, its advantage over the standard recipe is not statistically significant, and it incurs a cost of 1.7 times as many state-tree bit flips. Therefore, we report the standard recipe as our primary result. Crucially, the majority of this performance gain stems from the dose rule and the layer selection mechanism, both of which were established prior to evaluating this specific model.

6 Quantum-Hardware Proof of Concept

We report a small proof of concept for executing a repair-derived Ising objective on an IBM Heron quantum processor using the quantum approximate optimization algorithm (QAOA) [15, 61]. This specific instance isolates eight candidate weights from a single row of layer 15 in Qwen3.5-0.8B, with all other variables in the row fixed to zero. The separately calibrated parameter $\gamma = 0.0844$ adheres to a single-weight threshold rule, and we set $\lambda = \lambda_0$ (Section B). With eight binary variables yielding $2^8 = 256$ possible assignments, exhaustive enumeration establishes the exact optimum at negligible computational cost.

We optimized the QAOA angles classically using exact statevector simulation. In a single hardware execution, the depth-two QAOA circuit sampled the optimum in 110 out of 1,024 shots, compared to merely four shots for a uniform control. We note that uniform sampling serves as a relatively weak baseline for assessing optimization performance; matched random-angle circuits and classical solvers applied to identical instances would provide a more rigorous comparison. Full circuit configurations, sampling results, and resource accounting are detailed in Section B.

Crucially, this experiment establishes that a small, separately calibrated instance can be successfully encoded, executed, and sampled on quantum hardware. However, it does not constitute evidence of a quantum advantage. As demonstrated in Section 4, simulated annealing, simulated bifurcation, and the closed-form continuous edit all yield statistically indistinguishable accuracy, indicating that the current row-level problems are not computationally challenging for classical solvers, nor do marginal improvements in row energy translate to superior model performance. The primary value of these problems for quantum optimization lies in their role as a source of structured, real-world Ising instances equipped with a meaningful downstream metric: the accuracy of the repaired model. Significantly harder formulations, such as joint move selection or explicit bit budgets (Section C), would be required before a quantum solver could offer a tangible advantage. Consequently, the full-model accuracy results reported in Sections 4 and 5 are derived entirely from the classical GPU solver. A subsequent investigation into hybrid refinement would condition each block on the GPU solution, apply the established repair calibration, and evaluate the resulting checkpoint on standard benchmarks.

7 Related work

Weight-space programming and model editing. Takehi et al. [55] investigated whether a general-purpose LLM can read and write network weights; among other tasks, their agent successfully repairs the reasoning loops of Qwen3.5-0.8B. We study the identical repair task but replace the LLM agent with an optimizer built from the same empirical evidence. Classical model editing typically modifies factual associations through targeted updates of MLP weights [41, 42], locates knowledge within individual neurons [11], learns editors that predict weight updates [43], or combines fine-tuning differences [27]. Crucially, these methods produce continuous updates. In contrast, TREE-ISING selects sparse, discrete changes that the stored weight format can represent exactly, and Section 4.5 compares our approach with a ROME-like rank-one solution of our own objective. Furthermore, while bit-flip attacks demonstrate that a few flipped bits can substantially alter a network's behavior [50], we leverage this same sensitivity to repair rather than to harm.

Activation steering and directional interventions. Steering methods typically add directions to the residual stream at inference time [38, 56, 62], and directional ablation can be folded directly into the weights [5]. Our directions are contrasts of the same kind, measured at the onset of loops and at the conclusion of successful thinking traces. However, our intervention takes the form of a sparse weight edit optimized against a collateral penalty, requiring no modifications at inference time. Mechanistic studies of transformer circuits strongly motivate reading such directions directly from the residual stream [14]. Closest to our setting, ThinkEdit [53] identifies a linear direction governing reasoning length, isolates a small set of attention heads driving overly short reasoning, and removes that direction from their output projections, altering approximately 0.2% of the parameters. We address the opposite failure mode—reasoning that never terminates—and edit MLP output projections through sparse discrete moves rather than a continuous projection.

Repetition and looping. Degenerate repetition has been extensively addressed through decoding heuristics [22] and modified training objectives [58, 59], and recent reports on reasoning models analyze how extended thinking traces interact with decoding configurations [12, 48]. In contrast, TREE-ISING alters neither the decoder nor the training process; it simply edits the weights once, entirely offline.

Weight formats. Block floating-point and microscaling formats share exponents within groups of weights [13, 51], and lossless BF16 compression exploits the low entropy of these exponents [60]. State trees [3] provide a general representation of neural networks as paths of shared discrete decisions, with lossless BF16 storage and low-bit compression serving as specific instances; that work also heals a compressed model using gradient and curvature estimates. In our formulation, the state tree fundamentally defines the admissible moves of a forward-only objective.

QUBO formulations of network weights. AdaRound derives a binary rounding problem from a second-order approximation of the task loss [46], and Muñiz Subiñas et al. [45] formulate layer-output reconstruction for quantization as an exact QUBO that separates into independent output rows, solving it via simulated annealing. Layer-output reconstruction also underpins many pruning and quantization methods [16, 19]. Our objective augments this reconstruction term with a behavioral term along a measured direction and a sparsity cost, operating over moves that preserve the stored format intact. Formulated as Ising models, these row problems connect seamlessly to the broad toolbox of Ising formulations [40], simulated annealing [33], quantum annealing [28, 30], QAOA [15], and digital or analogue Ising machines [17, 44].

Quantum hardware for neural networks and LLMs. Quantum annealers have been utilized to train binary neural networks [52] and to select pruning and quantization configurations for convolutional networks [57]. For LLMs, recent work integrates variational quantum circuits into a pretrained model, either as parameter-efficient fine-tuning modules executed on a superconducting processor [34] or as unitary adapters inside the projection layers of Llama 3.1 8B running on IBM hardware [2]. In those studies, the quantum circuit constitutes a novel addition to the model architecture. In TREE-ISING, however, the model form remains entirely unchanged: the quantum device, much like the GPU, simply solves for which of the model's existing BF16 weights should be modified, as demonstrated here on a single eight-variable subproblem.

8 Limitations

Scope. Our study addresses a single failure mode: reasoning loops, specifically within two models of a single family that share a hybrid linear/full-attention architecture, evaluated exclusively on mathematical benchmarks. We have not assessed instruction-following capabilities, although Takehi et al. [55] report that the agent edit raised IFEval from 21.0% to 62.3%, nor have we tested general language modeling or non-mathematical benchmarks. Consequently, side effects remain unmeasured beyond the collateral term, which protects the layer's output only on tokens derived from the practice problems. Evaluating larger models, alternative model families, and other behavioral failure modes remains a direction for future work.

Baselines and Evaluation. The base model's 7.9% accuracy on MATH-500 is the figure reported by Takehi et al. [55]; we did not independently rerun the base model in our evaluation harness. Furthermore, the continuous edits detailed in Section 4.5 were evaluated exclusively on the 128-problem practice screen. Early exploratory variants on Qwen3.5-0.8B were screened on 100 MATH-500 problems, meaning the full MATH-500 results include problems exposed during development; however, the performance gap remains consistent on the 400 strictly held-out problems (Section 4.2). Each benchmark evaluation utilized a single generation seed. The p-values reported in the tables are nominal; notably, only the gains on MATH-500@4 and GSM8K over the agent edit survive a strict Bonferroni correction across the twelve agent comparisons.

Design Choices. Our repair recipe was formulated retrospectively after analyzing the agent's public logs. On Qwen3.5-0.8B, the edit utilizes the agent's selected layers and a dose 1.6 times that of the agent's, calibrated on the practice screen; notably, no strictly dose-matched edit was executed. While the Qwen3.5-2B experiment eliminates both of these dependencies, the rule excluding the final layer was established after inspecting the 0.8B scan. Furthermore, the layers selected in both models are full-attention blocks, suggesting that this agreement may stem from their shared architectural design rather than independent validation. For Qwen3.5-2B, greedy decoding and the GSM8K benchmark were selected based on base-model results from a 200-problem GSM8K test subset where the base model struggles most, whereas all edit-specific settings were determined solely on the practice problems.

Surrogate Objective and Solvers. Our objective functions as a first-order surrogate derived from the statistics of the unedited model: it inherently ignores how an edit in one layer alters the inputs to subsequent layers, and the dose serves merely as a linear proxy for the desired behavioral shift. What remains mathematically exact is the decomposition of this surrogate into independent row problems once candidate columns and a single move per weight are fixed; however, these two choices, along with the row solvers themselves, are heuristic. The practice screen resolves performance differences of approximately four percentage points, leaving the optimal dose and move set somewhat uncertain. Each solver was executed only once with a single seed, and their run times were not measured under strictly matched conditions; therefore, we make no definitive claims regarding their relative computational speed.

Accounting. Our state-tree bit counts assume a straightforward encoder and deliberately exclude modifications within the payloads of existing escapes; furthermore, direct patching of a compressed checkpoint has not yet been implemented. The reported time and cost metrics cover a single execution of a fixed recipe at list prices and explicitly exclude method development costs on both sides. Stage timings are derived from single runs executed with two versions of the solver script and should not be interpreted as a rigorous scaling measurement.

Quantum Hardware. The quantum hardware evaluation is confined to a single eight-variable subproblem, encompassing just 8 of the 7.3 million weights across the two edited matrices, with the remainder of its row fixed to zero. This evaluation utilized a dose calibration declared separately from the published edit, angles optimized classically, a single hardware job, and one calibration period. Crucially, the resulting solution was not written back into a model. The control baseline sampled uniformly; a random-angle control on the identical compiled circuits has not been executed. Consequently, this result demonstrates a correct mapping and objective-consistent sampling on quantum hardware, not a computational advantage over classical solvers. Furthermore, Equation (14) bounds the theoretical benefit of any faster solver within the confines of our current recipe.

9 Conclusion

While a frontier LLM can repair a looping model by directly editing its weights, we have demonstrated that this same repair task can be formulated as a sparse discrete optimization problem and solved in a matter of minutes. Specifically, TREE-ISING restricts admissible edits to sign flips and exponent shifts within the state tree of each weight group, evaluates them using forward statistics, and efficiently solves one small Ising problem per output row. On Qwen3.5-0.8B, our method produces an edit that is more accurate than the agent's baseline while modifying fewer weights and far fewer bits, all for a computational cost of approximately $0.90 per run. Crucially, both simulated annealing and the quantum-inspired simulated bifurcation solver yield repairs of equivalent quality, demonstrating that the performance gains stem fundamentally from the mathematical formulation itself rather than the tuning of a specific solver. In contrast, a dense closed-form solution of the identical objective achieves comparable accuracy only by altering nearly every weight across both matrices. Furthermore, when applied to Qwen3.5-2B—with target layers and dosage calibrated entirely from the base model—this same recipe elevates GSM8K accuracy under greedy decoding from 21.6% to 69.4%.

Finally, an eight-variable QAOA experiment executed on IBM quantum hardware serves as a preliminary proof of concept for running a repair-derived Ising instance on a quantum processor. Extending this validation to full-layer repairs and rigorously measuring the resulting downstream model accuracy remains a compelling direction for future work.

Code Availability. The source code implementing TREE-ISING will be made publicly available shortly at https://github.com/TheQuantumGOAT/Tree-Ising.

References

[1] AMC23: Problems of the 2023 American Mathematics Competitions. https://huggingface.co/datasets/math-ai/amc23, 2023. Evaluation split used by the agent's harness.

[2] B. Aizpurua, S. Singh, A. Kshetrimayum, S. S. Jahromi, and R. Orús. Quantum-enhanced large language models on quantum hardware via Cayley unitary adapters. *arXiv preprint arXiv:2605.05914*, 2026.

[3] A. Ali and S. Lefkimmiatis. Every weight is a path: Representing neural networks as trees of discrete decisions, 2026. arXiv preprint arXiv:2610.31931.

[4] American Compute. A100 40GB cloud pricing across providers. https://www.amcompute.com/gpu-cloud-prices/a100-40gb, 2026. Median on-demand price, October 2026.

[5] A. Arditi, O. Obeso, A. Syed, D. Paleka, N. Panickssery, W. Gurnee, and N. Nanda. Refusal in language models is mediated by a single direction. In *Advances in Neural Information Processing Systems*, 2024.

[6] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[7] D-Wave Quantum. Minor-embedding best practices. https://docs.dwavequantum.com/en/latest/quantum_research/embedding_guidance.html, 2026. Official documentation, accessed 9 November 2026.

[8] D-Wave Quantum. Per-QPU solver properties and schedules. https://docs.dwavequantum.com/en/latest/quantum_research/solver_properties_specific.html, 2026. Official documentation, accessed 9 November 2026.

[9] D-Wave Quantum. Ocean samplers and DWaveCliqueSampler. https://docs.dwavequantum.com/en/latest/ocean/api_ref_system/samplers.html, 2026. Official documentation, accessed 9 November 2026.

[10] D-Wave Quantum. Operation and timing. https://docs.dwavequantum.com/en/latest/quantum_research/operation_timing.html, 2026. Official documentation, accessed 9 November 2026.

[11] D. Dai, L. Dong, Y. Hao, Z. Sui, B. Chang, and F. Wei. Knowledge neurons in pretrained transformers. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pages 8493–8502, 2022. doi: 10.18653/v1/2022.acl-long.581.

[12] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[13] M. Drumond, T. Lin, M. Jaggi, and B. Falsafi. Training DNNs with hybrid block floating point. In *Advances in Neural Information Processing Systems*, 2018.

[14] N. Elhage, N. Nanda, C. Olsson, T. Henighan, N. Joseph, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.

[15] E. Farhi, J. Goldstone, and S. Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.

[16] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.

[17] H. Goto, K. Tatsumura, and A. R. Dixon. Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems. *Science Advances*, 5(4):eaav2372, 2019.

[18] H. Goto, K. Endo, M. Suzuki, Y. Sakai, T. Kanao, Y. Hamakawa, R. Hidaka, M. Yamasaki, and K. Tatsumura. High-performance combinatorial optimization based on classical mechanics. *Science Advances*, 7(6):eabe7953, 2021.

[19] B. Hassibi and D. G. Stork. Second order derivatives for network pruning: Optimal brain surgeon. In *Advances in Neural Information Processing Systems*, 1993.

[20] C. He, R. Luo, Y. Bai, S. Hu, Z. L. Thai, J. Shen, J. Hu, X. Han, Y. Huang, Y. Zhang, et al. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2024.

[21] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021.

[22] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020.

[23] IBM Quantum. Fez real-backend configuration snapshot. https://quantum.cloud.ibm.com/docs/en/api/qiskit-ibm-runtime/fake-provider-fake-fez, 2026. Configuration distributed with qiskit-ibm-runtime 0.51.0, reference backend version 1.2.8.

[24] IBM Quantum. Hardware and roadmap. https://www.ibm.com/quantum/hardware, 2026. Accessed 9 November 2026.

[25] IBM Quantum. Processor types. https://quantum.cloud.ibm.com/docs/en/guides/processor-types, 2026. Accessed 9 November 2026.

[26] IBM Quantum. IBM quantum technology. https://quantum.cloud.ibm.com/learning/en/courses/quantum-business-foundations/quantum-technology, 2026. Accessed 9 November 2026.

[27] G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations*, 2023.

[28] M. W. Johnson, M. H. S. Amin, S. Gildert, T. Lanting, F. Hamze, N. Dickson, R. Harris, A. J. Berkley, J. Johansson, P. Bunyk, et al. Quantum annealing with manufactured spins. *Nature*, 473:194–198, 2011.

[29] T. Jones and J. Gacon. Efficient calculation of gradients in classical simulations of variational quantum algorithms. *arXiv preprint arXiv:2009.02823*, 2020.

[30] T. Kadowaki and H. Nishimori. Quantum annealing in the transverse Ising model. *Physical Review E*, 58(5):5355, 1998.

[31] D. Kalamkar, D. Mudigere, N. Mellempudi, D. Das, K. Banerjee, S. Avancha, D. T. Vooturi, N. Jammalamadaka, J. Huang, H. Yuen, et al. A study of BFLOAT16 for deep learning training. *arXiv preprint arXiv:1905.12322*, 2019.

[32] A. Khaliq. Independent re-run of the Qwen3.5-0.8B weight-space programming checkpoint on the full MATH-500. Hugging Face model card, https://huggingface.co/RikiyaT/Qwen3.5-0.8B-weight-space-programming, 2026. Accessed 9 November 2026.

[33] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[34] X. Kong, L. Li, Z. Chen, C. Xue, X. Xu, H. Liu, Y. Wu, Y. Fang, H. Fang, K. Chen, Y. Yang, M. Dou, and G. Guo. Quantum-enhanced LLM efficient fine tuning. *arXiv preprint arXiv:2503.12790*, 2025.

[35] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM Symposium on Operating Systems Principles*, 2023.

[36] H. Kydlíček and Hugging Face. Math-Verify: A robust mathematical expression evaluation system. https://github.com/huggingface/Math-Verify, 2025.

[37] A. Lewkowycz, A. Andreassen, D. Dohan, E. Dyer, H. Michalewski, V. Ramasesh, A. Slone, C. Anil, I. Schlag, T. Gutman-Solo, et al. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems*, 2022.

[38] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. In *Advances in Neural Information Processing Systems*, 2023.

[39] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, 2024.

[40] A. Lucas. Ising formulations of many NP problems. *Frontiers in Physics*, 2:5, 2014.

[41] K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022.

[42] K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. In *International Conference on Learning Representations*, 2023.

[43] E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. In *International Conference on Learning Representations*, 2022.

[44] N. Mohseni, P. L. McMahon, and T. Byrnes. Ising machines as hardware solvers of combinatorial optimization problems. *Nature Reviews Physics*, 4:363–379, 2022.

[45] S. Muñiz Subiñas, M. L. González, J. Ruiz Gómez, A. Mata Ali, J. Martínez Martín, M. Franco Hernando, Á. M. García-Vico, and J. Sedano. Optimization of the quantization of dense neural networks from an exact QUBO formulation. *arXiv preprint arXiv:2510.16075*, 2025.

[46] M. Nagel, R. A. Amjad, M. Van Baalen, C. Louizos, and T. Blankevoort. Up or down? Adaptive rounding for post-training quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *PMLR*, pages 7197–7206, 2020.

[47] OpenAI. OpenAI API pricing. https://developers.openai.com/api/docs/pricing, 2026. GPT-6 Astra, standard tier; accessed November 2026.

[48] Qwen Team. Qwen3.5-0.8B model card. https://huggingface.co/Qwen/Qwen3.5-0.8B, 2026.

[49] Qwen Team. Qwen3.5-2B model card. https://huggingface.co/Qwen/Qwen3.5-2B, 2026.

[50] A. S. Rakin, Z. He, and D. Fan. Bit-flip attack: Crushing neural network with progressive bit search. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019.

[51] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

[52] M. Sasdelli and T.-J. Chin. Quantum annealing formulation for binary neural networks. *arXiv preprint arXiv:2107.02751*, 2021.

[53] C.-E. Sun, G. Yan, and T.-W. Weng. ThinkEdit: Interpretable weight editing to mitigate overly short thinking in reasoning models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 17001–17025, 2025.

[54] R. Takehi, J. Teoh, A. Zhang, and O. Khattab. lm_understand-logs: Complete logs of the weight-space programming experiments. https://github.com/RikiyaT/lm_understand-logs, 2026.

[55] R. Takehi, J. Teoh, A. Zhang, and O. Khattab. Sparks of weight-space programming: LLMs can natively read & write neural network weights. https://rikiyat.github.io/blog/weight-space-programming/, 2026. Blog post, MIT CSAIL, 2 November 2026.

[56] A. M. Turner, L. Thiergart, G. Leech, D. Udell, J. J. Vazquez, U. Mini, and M. MacDiarmid. Steering language models with activation engineering. *arXiv preprint arXiv:2308.10248*, 2023.

[57] Z. Wang, B. C. M. Choong, T. Huang, D. Gerlinghoff, R. S. M. Goh, C. Liu, and T. Luo. Is quantum optimization ready? An effort towards neural network compression using adiabatic quantum computing. *arXiv preprint arXiv:2505.16332*, 2025.

[58] S. Welleck, I. Kulikov, S. Roller, E. Dinan, K. Cho, and J. Weston. Neural text generation with unlikelihood training. In *International Conference on Learning Representations*, 2020.

[59] J. Xu, X. Liu, J. Yan, D. Cai, H. Li, and J. Li. Learning to break the loop: Analyzing and mitigating repetitions for neural text generation. In *Advances in Neural Information Processing Systems*, 2022.

[60] T. Zhang, Y. Sui, S. Zhong, V. Chaudhary, X. Hu, and A. Shrivastava. 70% size, 100% accuracy: Lossless LLM compression for efficient GPU inference via dynamic-length float. *arXiv preprint arXiv:2504.11651*, 2025.

[61] L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin. Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices. *Physical Review X*, 10:021067, 2020. doi: 10.1103/PhysRevX.10.021067.

[62] A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, R. Ren, A. Pan, X. Yin, M. Mazeika, A.-K. Dombrowski, et al. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.

A Implementation details

Every layer reaches at least its target dose, overshooting it by 0.3 to 2.3% (SB by 0.3 and 0.9%), and only layer 19 of Qwen3.5-2B needed a second value of λ to meet its budget. The two annealing times in Table 8 come from different versions of the solver script, the second a generic reimplementation used for Qwen3.5-2B, run at different times; they include model loading and a γ search whose length varies, and should not be read as a scaling measurement.

Hardware interpretation. Each row objective has the Ising form used by quantum annealers and other specialized optimization machines. Sections B.5 and B.6 describe the conditional-block route to hardware and the evaluation protocol. All reported checkpoints were built on one GPU, with simulated annealing or, where marked SB, with simulated bifurcation.

B Quantum hardware experiment details

B.1 Instance and calibration

The block statistics were computed from 224 looping traces and 33 correct traces. They include 195,013 loop tokens and 567,053 protected tokens per layer, together with 2,112 concluding and 120,068 continuing tokens from the correct traces. We downloaded only the layer-15 and layer-19 `down_proj` tensors from the public Qwen3.5-0.8B checkpoint (revision `2fc0636`); their byte ranges and hashes are documented in the reproducibility package. We retained $K = 512$, the eight moves in Equation (1), $\beta = 0.5$, and the sparsity scale

$$\lambda = 2\,\mathrm{median}_{r,a,\,f\in\{-1,2,\frac{1}{2}\}}\; C_{\mathscr{P},j_a j_a}\big[(f-1)W_{rj_a}\big]^2. \tag{8}$$

Since the calibrated γ used for the published edit was not saved with the statistics, we specified a separate calibration rule before examining any solution. For each candidate weight, we computed the single-weight entry threshold

$$t_{ra} = \min_{f\ \text{admissible},\ \ell_{raf}<0} \frac{C_{\mathscr{P},j_a j_a}\,\delta_{raf}^2 + \lambda}{-\ell_{raf}}, \qquad \ell_{raf} = u_r\,\mu_{\mathscr{R},j_a}\,\delta_{raf}, \tag{9}$$

which marks the point at which activating that weight alone becomes energetically favorable. We set γ to $(1+10^{-6})$ times the 65,536th smallest of the 505,622 finite positive thresholds, obtaining

$$\gamma = 0.0844304006, \qquad \lambda = 5.88178264\times 10^{-7}.$$

We then selected layer 15 and the row with the largest $|u_r|$ (row 5), restricting the block to its first eight candidate columns, $(0, 46, 1184, 294, 371, 504, 264, 1130)$, while fixing all other variables in that row to zero. The block was constructed in float64 from the exact BF16 weights. For all 256 assignments, its energy agrees with a direct evaluation of the dose, collateral, and sparsity terms to within 7.05×10^{-19}. Exhaustive search identifies the unique optimum

$$x^\star = (1,1,0,0,1,0,1,1), \qquad \mathscr{E}_B(x^\star) = -7.68966437\times 10^{-4}. \tag{10}$$

Qiskit displays this bitstring as `11010011`, reflecting its ordering of the bits as $x_7\ldots x_0$.

B.2 Circuits

Under Qiskit's convention $Z_i = 1-2x_i$, the cost Hamiltonian associated with $\mathscr{E}_B(x) = h^\top x + x^\top J x$ is

$$\widehat{H}_C = \sum_i \Big(-\frac{h_i}{2} - \frac{1}{2}\sum_j J_{ij}\Big) Z_i + \sum_{i<j}\frac{J_{ij}}{2} Z_i Z_j, \qquad E_0 = \frac{1}{2}\sum_i h_i + \frac{1}{4}\sum_{ij} J_{ij}, \tag{11}$$

where adding E_0 to each eigenvalue recovers $\mathscr{E}_B$ for all 256 assignments. Defining $D = \mathrm{diag}(\mathscr{E}_B(x)/s)$ with $s = 7.32314592\times 10^{-5}$ and using the mixer $B = \sum_j X_j$, we write the depth-p state as

$$|\psi_p(\boldsymbol{\vartheta},\boldsymbol{\varphi})\rangle = e^{-i\varphi_{p-1}B}e^{-i\vartheta_{p-1}D}\cdots e^{-i\varphi_0 B}e^{-i\vartheta_0 D}\,|+\rangle^{\otimes 8}. \tag{12}$$

Table 8: Experimental settings for the two model repairs. Reported stage runtimes were measured on a single NVIDIA A100 40 GB GPU.

Setting	Qwen3.5-0.8B (Section 4)	Qwen3.5-2B (Section 5)
Edit configuration		
Target tensors	`down_proj`, layers 15 and 19	`down_proj`, layers 15 and 19. The stronger variant also includes layer 14
Tensor dimensions	$1{,}024 \times 3{,}584$ per layer	$2{,}048 \times 6{,}144$ per layer
Layer selection	Agent-selected layers, corroborated by a forward scan	Forward scan, excluding the final layer
Candidate neurons	$K = 512$ of 3,584, with the same candidate columns across all rows within a layer	$K = 512$ of 6,144 per row
Allowed moves	Eight multiplicative factors from Equation (1)	Same
Edit direction	$u \propto u_{\text{loop}} - 0.5\,u_{\text{end}}$, computed per layer	Same
Dose target	$1.6\times$ the agent edit's dose along u. Equivalent to $6.7\times$ and $6.1\times$ the model's own write in layers 15 and 19, respectively	$6.5\times$ the model's own write. Stronger variant uses $9\times$
Edit budget	65,536 changed weights per layer	200,000 changed weights. Stronger variant permits 300,000
Statistics collection		
Collection data	384 practice problems, comprising 192 MATH-train and 192 GSM8K-train problems, with one completion each	Same problems
Decoding	Agent protocol, temperature 0.6	Greedy
Token cap	8,192	8,192
Collected traces	224 looping, 33 correct	275 looping, 65 correct
Tokens per layer	$\lvert\mathscr{R}\rvert = 195$k, $\lvert\mathscr{P}\rvert = 567$k, $\lvert\mathscr{Z}\rvert = 2.1$k, $\lvert\mathscr{M}\rvert = 120$k	$\lvert\mathscr{R}\rvert = 246$k, $\lvert\mathscr{P}\rvert = 655$k, $\lvert\mathscr{Z}\rvert = 4.2$k, $\lvert\mathscr{M}\rvert = 127$k
Optimization		
Simulated annealing	16 replicas and 300 sweeps. Temperature decreases from 2 to $10^{-4}\times$ the median $\lvert h\rvert$. Calibration uses 4 replicas and 60 sweeps	Same
Simulated bifurcation	Discrete SB with 1,200 steps of size 0.5. Pump ramp over the first 25% of steps, global field scaling, and best-state tracking every 20 steps. Followed by float64 single-bit descent with at most 1,024 flips. Batches of 32 rows, with 16 replicas for the final solve and 4 for calibration	Not run
Selection and evaluation		
Selection data	128 held-out practice problems, comprising 96 MATH and 32 GSM8K problems	Same problems
Selection decoding	One completion per problem, agent protocol, 32k-token cap	One completion per problem, greedy, 8k-token cap
Evaluation protocol	Shared protocol described below	Shared protocol, with greedy decoding and an 8k-token cap
Stage runtime (minutes)		
Generation	5.7	5.9
Layer scan	—	3.6
Statistics	5.1	4.8
Simulated annealing	8.8	6.7
Selection screen	11.0	5.0

Shared evaluation protocol. Evaluation uses vLLM 0.32 with BF16 precision and thinking enabled. Each problem is followed by "Please reason step by step, and put your final answer within \boxed{}." The last boxed answer after `</think>` is graded using Math-Verify.

Table 9: Dose calibration of every edited layer. Own write is $|u^\top W \mu_{\mathscr{R}}|$ of the base model; doses are $u^\top \Delta\mu_{\mathscr{R}}$ after rounding to BF16. λ attempts counts the values of λ tried, with a full γ search at each.

Model	Layer	Own write	Agent dose	Target dose	Achieved dose	Achieved / own	λ attempts	Changed weights
Qwen3.5-0.8B	15	0.328	−1.364	−2.183	−2.225	6.8	1	51,754
Qwen3.5-0.8B	19	0.498	−1.910	−3.056	−3.075	6.2	1	60,876
Qwen3.5-0.8B, SB	15	0.328	−1.364	−2.183	−2.189	6.7	1	48,860
Qwen3.5-0.8B, SB	19	0.498	−1.910	−3.056	−3.083	6.2	1	59,467
Qwen3.5-2B	15	1.035	–	−6.730	−6.884	6.6	1	111,211
Qwen3.5-2B	19	1.755	–	−11.405	−11.522	6.6	2	181,634

We optimize the angles by minimizing the exact statevector mean $\langle\psi_p|D|\psi_p\rangle$ using L-BFGS-B with analytic adjoint gradients [29], subject to $\vartheta_\ell \in [0, 2\pi]$ and $\varphi_\ell \in [0, \pi]$. At each depth, we use 48 starts, including states inherited from the preceding depth, split cost and mixer layers, schedules interpolated in the spirit of parameter transfer [61], perturbed continuations, and random angles. Each start is limited to 800 iterations and 4,000 evaluations, with seed 17. Retaining the best state encountered ensures that the ideal mean energy does not increase with depth.

We restricted the live IBM Fez target to the eight physical qubits used in the first run. At each depth, we tried four compiler seeds (17 to 20) at optimization level 3 with exact synthesis. We retained the compilation with the fewest CZ gates, breaking ties by selecting the smallest circuit depth. Gate twirling, measurement twirling, and dynamical decoupling were disabled. Each QAOA circuit was paired with a uniform-sampling control measured using the same final readout assignment.

Table 10: Device and execution settings. Circuit statistics are taken from the archived submitted circuits.

Backend	`ibm_fez`, IBM Heron r2, 156 fixed-frequency transmon qubits with tunable couplers and heavy-hexagonal connectivity [24–26]
Access	IBM Quantum Platform Open Plan, `us-east`, job mode, using Qiskit 2.6.2 and IBM runtime client 0.51.0
Physical qubits	121, 122, 123, 124, 125, 136, 142, 143
Problem	8 binary variables and 28 nonzero logical couplings
Shots	1,024 per circuit, with six circuits (three QAOA depths and their controls) submitted in one job
Compiled QAOA circuits	$p = 1$ has 107 CZ gates and depth 211, $p = 2$ has 220 CZ gates and depth 387, and $p = 3$ has 350 CZ gates and depth 617
Reference device graph	`conf_fez.json` from qiskit-ibm-runtime 0.51.0, backend version 1.2.8, with 176 undirected connections [23]

Table 11: Physical qubit assignments. Classical bit j records variable x_j. Each control uses the final readout assignment of its corresponding QAOA circuit.

Variable	x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7
Circuit entry	123	124	122	136	125	121	143	142
Readout, $p = 1$	142	143	121	122	136	123	125	124
Readout, $p = 2$	125	121	124	122	136	123	142	143
Readout, $p = 3$	142	143	121	122	136	125	123	124

B.3 Hardware sampling results

The observed optimum rate was highest at depth two (10.74%), followed by depth three (5.37%) and depth one (3.22%) (Table 13). The ideal statevector probability of the optimum increases with depth, whereas the measured probability

Table 12: Classically optimized angles in radians, listed in increasing layer order. Evaluation counts and times are cumulative, as optimization at each depth builds on the preceding depth.

p	Cost angles $\boldsymbol{\vartheta}$	Mixer angles $\boldsymbol{\varphi}$	Evaluations	Time (s)
1	(0.474617)	(2.772206)	769	0.115
2	(0.462255, 0.650097)	(2.599518, 2.662857)	1,975	0.430
3	(0.342703, 0.641411, 0.645390)	(2.654418, 2.681259, 2.847049)	3,496	1.011

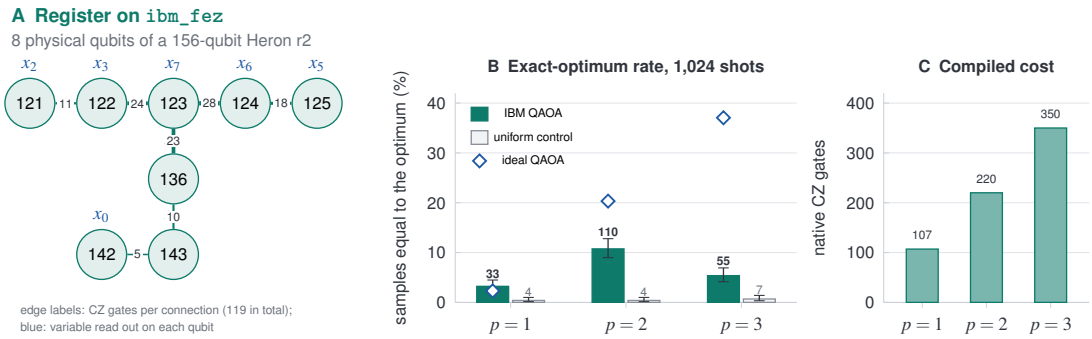


Figure 5: A statistics-derived repair block on IBM quantum hardware. (A) The eight-variable block runs on eight connected qubits of `ibm_fez`; edge labels are the CZ counts of the earlier $p = 1$ diagnostic (119 CZ in total, Section B), not of the 107-CZ circuit in (B) and (C). (B) Share of the 1,024 shots that equal the unique exact optimum, with Wilson 95% intervals, for QAOA at depths one to three and for matched uniform controls; diamonds give the ideal noiseless rate. (C) Two-qubit gate count after compilation. Depth two gives the best measured rate; at depth three gate noise outweighs the higher ideal rate.

Table 13: Eight-variable QAOA proof of concept on IBM Heron (`ibm_fez`, one job, 1,024 shots per circuit). The optimum rate is the fraction of samples matching the unique exact optimum, reported with a binomial Wilson 95% interval. Ideal rates are exact statevector probabilities at the same angles. Energies are multiplied by 10^4. Lower values are better, and the optimum is -7.690.

Circuit	Native CZ	Optimum hits	Optimum rate (%) [95% interval]	Ideal rate (%)	Mean energy $\pm$ SE
QAOA, $p = 1$	107	33	3.22 [2.30, 4.49]	2.31	-5.617 ± 0.040
QAOA, $p = 2$	220	**110**	**10.74** [8.99, 12.79]	20.33	$\mathbf{-5.859 \pm 0.047}$
QAOA, $p = 3$	350	55	5.37 [4.15, 6.93]	37.06	-5.756 ± 0.046
Uniform control, $p = 1$	0	4	0.39 [0.15, 1.00]	0.39	-4.476 ± 0.048
Uniform control, $p = 2$	0	4	0.39 [0.15, 1.00]	0.39	-4.454 ± 0.050
Uniform control, $p = 3$	0	7	0.68 [0.33, 1.40]	0.39	-4.448 ± 0.047

peaks at depth two. These measurements were obtained in a single job during one calibration period. The uniform controls contain zero native CZ gates, compared with 107 to 350 in the QAOA circuits. Matched circuits with random angles would provide a stronger control for assessing the effect of the optimized angles.

B.4 Resources and an earlier diagnostic

The exact ideal mean energies at the three depths are $(-5.636507, -6.727090, -7.120713) \times 10^{-4}$, compared with the minimum -7.689664×10^{-4}. IBM reports 4 seconds of QPU time for the six circuits and 6,144 shots (job `dbouna8ou4as73csnpd0`, 9 November 2026). According to the service timestamps, 2,648.9 seconds elapsed between creation and completion, including 2,352.7 seconds before the job entered the running state. Queueing therefore accounts for most of the elapsed time. Classical preparation took 61 seconds in total.

An earlier single-circuit diagnostic (job `dborrfs4148s739tgkig`) evaluated the same block at $p = 1$ under a different compilation policy, yielding 119 CZ gates and depth 254. The optimum was sampled 21 times in 1,024 shots (2.05%, Wilson interval [1.35, 3.12]%), compared with 6 times (0.59%, [0.27, 1.27]%) for its uniform control. The corresponding ideal rates were 2.31% and 0.39%, and the reported QPU time was 3 seconds. The edge counts

in Figure 5 correspond to the 119 CZ gates from this earlier run. The package accompanying the paper includes the QUBO, calibration metadata, compiled circuits, raw counts, job identifiers, input hashes, and scripts.

B.5 Prospective extension to larger repair blocks

A dense row of $K = 512$ variables contains $K(K-1)/2 = 130{,}816$ pairwise couplings. Minor embedding requires a physical connection for each logical coupling, but the working graph of D-Wave's `Advantage2_system1` lists only 41,205 couplers in total [8], ruling out an embedding of the full row. Dense embeddings on current annealers accommodate at most a few hundred variables [7, 9]. A future hardware backend could therefore operate on conditional blocks. Let $\mathscr{E}_r(x) = h^\top x + x^\top J x$, where J is symmetric with a zero diagonal. For a variable block B, fixing its complement F at the current solution x_F gives the block problem

$$\mathscr{E}_B(z) = z^\top J_{BB}\, z + \left(h_B + 2J_{BF}x_F\right)^\top z + \text{const} \tag{13}$$

which is again a QUBO. Its effective field incorporates the influence of every fixed variable. A hybrid loop would start from the GPU solution, submit difficult blocks to the quantum device, and accept updates only when they decrease the full row energy. The classical component would handle the calibration of γ and λ. Submission counts also matter. A single pass over disjoint blocks of size b requires $2{,}048\lceil 512/b\rceil$ submissions, amounting to 16,384 at $b = 64$, before any repeated sweeps or calibration.

B.6 Comparison protocol for future hardware extensions

Hardware and classical solvers should be evaluated on identical, archived instances, using the same statistics, candidate moves, penalties, blocks, fixed assignments, and starting states. Reports should include the achieved energy, its gap to the best known or certified optimum, feasibility, coefficient precision, chain breaks, and the complete time budget. Timing should distinguish programming, annealing, and readout [10] from embedding, data transfer, and queueing. The end-to-end evaluation should measure the repaired checkpoint's accuracy on fresh problems.

The stage times in Table 4 also limit the speedup available from a faster solver. The recipe requires 21.8 minutes of model computation and 8.8 minutes of solver time. If the solver took no time at all, the maximum speedup would be

$$S_{\max} = \frac{21.8 + 8.8}{21.8} \approx 1.40. \tag{14}$$

Assessing a future hardware extension would require matched-instance comparisons of both solution quality and end-to-end performance.

C Formulation extensions

The results in this paper use the single-move formulation of Section 3.4. This appendix records three extensions that enlarge the optimization problem, and with it the potential role of specialized solvers.

Joint choice among moves. The current formulation preselects one move per candidate weight before solving the row. Interactions can make a different move preferable once neighbouring weights change. Instead, let $i = (j,m)$ index a candidate weight j and an admissible move m, with $\delta_i = (f_m - 1)W_{rj}$ and $y_i \in \{0,1\}$. At most one move per weight may be active,

$$\textstyle\sum_m y_{jm} \le 1, \tag{15}$$

which a QUBO can enforce with a pairwise penalty $\rho \sum_j \sum_{m<m'} y_{jm} y_{jm'}$:

$$\widetilde{\mathscr{E}}_r(y) = \sum_i A_i y_i + \sum_{i<k} B_{ik} y_i y_k + \rho \sum_j \sum_{m<m'} y_{jm} y_{jm'}, \tag{16}$$

with

$$A_i = \gamma u_r \mu_{\mathscr{R},j_i}\delta_i + C_{\mathscr{P},j_i j_i}\delta_i^2 + \lambda + \eta\, b_i, \qquad B_{ik} = 2\,\delta_i \delta_k\, C_{\mathscr{P},j_i j_k}. \tag{17}$$

Here b_i is the bit cost of move i and $\eta \geq 0$ prices it. Any $\rho > \max_i \big(|A_i| + \sum_{k \neq i} |B_{ik}|\big)$ makes every violating assignment suboptimal, because removing one conflicting move lowers the penalty by at least ρ and changes the rest of the objective by less. Such a bound widens the coefficient range, which matters for analogue hardware; a categorical solver can keep one move or none per weight by construction instead. With eight moves the row grows to at most $8K$ variables.

Bit costs. For non-escaped weights whose moves keep the group maximum, the mantissa and the escape list, the state-tree cost is additive across weights,

$$b_i^{\text{tree}} = \mathbf{1}\big[s_i' \neq s_i\big] + \text{popcount}_4\big(D_i' \oplus D_i\big), \tag{18}$$

where s_i is the sign bit and $\oplus$ is bitwise exclusive or. Pricing these costs lets one budget written bits instead of changed weights. Moves that change a group's shared exponent can be included by charging the node and the states of the other weights in the group they shift; such moves are much more expensive in stored bits than tree-local ones.

Dose rules and budgets. Global dose or cardinality penalties couple the rows. Linear Lagrange penalties with an outer calibration loop, or explicit per-row budgets enforced by a categorical search, keep the rows independent. A sequential variant can refresh forward statistics after editing one layer, so that the next layer's objective sees the edited inputs, at the price of extra forward passes. Changes to the formulation and to the solver should be evaluated separately: first compare solvers on identical single-move instances, then compare formulations at matched runtime and edit budgets, and finally measure the repaired models on fresh problems and on unrelated tasks.